BASIC++ v6.5.2 DEVELOPER AND CONTRIBUTOR GUIDE

1. PURPOSE AND ARCHITECTURE

The BASIC++ developer guide defines the source-level interfaces, build structure, execution model, and extension procedures used to modify the BASIC++ language implementation. BASIC++ is organized as a modular interpreter and runtime system written in C17. Language processing is divided into lexical analysis, parsing, expression evaluation, runtime execution, virtual machine services, platform abstraction, and optional subsystem libraries.

The interpreter accepts BASIC source text and processes it through the lexer, parser, evaluator, and virtual machine. Lexical analysis converts source characters into temporary token objects. The parser consumes those tokens and constructs abstract syntax tree nodes representing statements and expressions. Runtime evaluation operates on the resulting AST representation and produces values represented by the BASIC++ value system. Statement execution is dispatched through the virtual machine.

The implementation is designed so that operating-system interfaces do not appear in the language engine. File systems, console devices, timing, process services, networking, dynamic libraries, and other host-dependent operations are provided through the platform abstraction layer. This permits the same engine code to be compiled for different host environments without introducing operating-system dependencies into the parser or evaluator.

The implementation uses C17 and must remain compatible with the ISO C17 language standard. Compiler-specific language extensions, pragmas, C++ source, and implementation-dependent language features are not permitted in the engine.

1. SOURCE ORGANIZATION

Language implementation source files are separated according to their responsibility. The lexer is responsible for converting source text into tokens. The parser converts tokens into AST nodes. The evaluator resolves expressions and produces BValue instances. Statement handlers implement BASIC statements. Function handlers implement built-in functions. The virtual machine executes the resulting operations.

A language component that requires platform services must access those services through libplatform. Direct inclusion of operating-system headers such as windows.h or unistd.h is prohibited in engine components.

The source tree uses a one-to-one relationship between language components and their implementation files where that convention is defined by the project. A keyword implementation must therefore remain associated with its designated source and header files rather than being distributed through unrelated generic modules.

1. VALUE REPRESENTATION

Runtime values are represented by BValue structures containing a type discriminator and an associated value representation. The discriminator must be inspected before accessing the corresponding union member.

Numeric values use VAL_NUMBER. String values use VAL_STRING and are managed through the BASIC++ string ownership system. Structured values such as arrays use their corresponding descriptors.

A BValue union member must never be accessed without first verifying the value type. Code that accesses .as.number when the discriminator does not contain VAL_NUMBER has undefined behavior. The same restriction applies to string and structured-value members.

String ownership is controlled through str_retain() and str_release(). Any function that acquires ownership of a reference is responsible for releasing that reference on both normal and error paths. A statement handler must not leak a temporary string when execution terminates because of an error.

1. LEXICAL ANALYSIS AND PARSING

The lexer exposes tokenization through lex_next(). Tokens identify lexical elements such as keywords, identifiers, numeric constants, and string constants. Typical token discriminators include TOK_KEYWORD, TOK_IDENT, TOK_NUMBER, and TOK_STRING.

Token objects are temporary parser data. The implementation does not permanently replace source storage with a tokenized representation. Tokens are consumed during parsing and released when they are no longer required.

The parser constructs ASTNode objects representing the syntactic structure of a BASIC statement or expression. Expression evaluation is performed after the parser has established the syntactic structure of the operation.

The VM does not depend on recursive host C calls for expression evaluation. Explicit evaluation state is maintained by runtime structures so that BASIC program nesting does not directly consume an equivalent amount of the host C call stack.

1. ADDING A NEW KEYWORD OR STATEMENT

Adding a BASIC++ statement requires changes to the lexer, statement implementation, VM registration, build system, and documentation.

The first change is the keyword identifier. Add the new keyword to BppKeywordId in engine/include/lexer/lexer.h.

```
typedef enum {
    /* Existing keyword identifiers. */

    KW_MYSTMT,
```

```
    KW_COUNT
} BppKeywordId;
```

The keyword text must then be registered in the lexer keyword table in engine/src/lexer/lexer.c.

```
static const char *k_keywords[] = {
    /* Existing keywords. */

    "MYSTMT",

    NULL
};
```

The enumeration entry and keyword table entry must remain synchronized. A mismatch between the identifier and its corresponding string can cause the lexer to return the wrong keyword identifier.

The statement implementation should have a dedicated public header and C source file.

```
#include "statements/custom/mystmt.h"
#include "vm/vm.h"
#include "eval/eval.h"
#include "types/errors.h"

BppError stmt_mystmt_exec(BppVM *vm, BppParserContext *pctx)
{
    BValue val;

    if (vm == NULL || pctx == NULL) {
        return ERR_NULL_POINTER;
    }

    val = eval_expression(pctx);

    if (val.type == VAL_ERROR) {
        return val.as.error;
    }

    /*
     * Perform statement-specific processing here.
     */

    if (val.type == VAL_STRING) {
        str_release(val.as.string);
    }
```

```
    return ERR_NONE;
}
```

The handler must validate its VM and parser context before accessing either structure. Expression results must be checked before use. Any owned string returned by expression evaluation must be released according to the ownership contract.

The statement must then be registered with the VM dispatcher.

```
#include "statements/custom/mystmt.h"

stmt_register(
    vm->stmt_reg,
    KW_MYSTMT,
    stmt_mystmt_exec,
    "MYSTMT",
    STMT_FLAG_BOTH
);
```

The implementation must also be added to the CMake build.

```
add_library(stmt_mystmt OBJECT
    src/statements/custom/stmt_mystmt.c
)

target_include_directories(stmt_mystmt PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)

set_target_properties(stmt_mystmt PROPERTIES
    C_STANDARD 17
    C_STANDARD_REQUIRED ON
)

target_sources(libcore PRIVATE
    $<TARGET_OBJECTS:stmt_mystmt>
)
```

If the statement belongs to the standard library rather than the foundational core, its object target is added to libstandard instead.

The statement must have corresponding entries in the runtime help registry and documentation tree. The normal locations are engine/src/runtime/help_data.h, help/MYSTMT.txt, and docs/MYSTMT.md.

1. ADDING A BUILT-IN FUNCTION

Built-in functions are evaluated as expressions. A function implementation receives the VM context, an argument array, and the argument count.

A function must validate the number and type of arguments before accessing them.

```
BValue func_myfn(BppVM *vm, BValue *args, int arg_count)
{
    double result;

    if (arg_count != 1) {
        return val_make_error(ERR_WRONG_NUMBER_OF_ARGS);
    }

    if (args[0].type != VAL_NUMBER) {
        return val_make_error(ERR_TYPE_MISMATCH);
    }

    result = args[0].as.number * 2.0;

    return val_make_number(result);
}
```

The function is registered through the expression dispatch system in engine/src/eval/dispatch.c. The dispatch table associates the BASIC function name with the C implementation.

A function must not assume that the caller supplied a valid type merely because the parser accepted the expression. Syntax validation and runtime type validation are separate operations.

1. MODIFYING EXISTING SYNTAX

BASIC++ provides both runtime and source-level mechanisms for modifying language behavior.

The ALIAS mechanism can provide an alternate name for an existing keyword.

```
ALIAS "ESCREVER" FOR "PRINT"
ESCREVER "OLA, MUNDO!"
```

Operator aliases can provide alternate textual representations of operators.

```
ALIAS OPERATOR "E" FOR "AND"
IF A = 1 E B = 2 THEN PRINT "SIM"
```

SCOPE HOOK can execute BASIC code around an existing statement.

```
SCOPE HOOK BEFORE "PRINT" GOSUB 5000
```

OVERRIDE can replace the normal execution routine for a statement.

```
OVERRIDE "PRINT" WITH GOSUB 9000
```

These facilities modify runtime behavior without requiring a recompilation of the engine when the requested change is supported by the active dialect.

Permanent changes to a standard keyword require modification of the corresponding statement implementation. SOURCE_MAP.md should be used to identify the implementation file. After modifying the handler, the affected executable must be rebuilt using CMake.

1. MODULAR BUILD ARCHITECTURE

BASIC++ uses an accumulative library architecture. The dependency direction begins with libboot and proceeds through platform services, kernel services, language execution, hardware services, server services, scripting, core language functionality, optional language extensions, standard workstation functionality, advanced graphics functionality, and external extensions.

libboot provides startup and shutdown sequencing.

libplatform provides operating-system abstraction services. Its interfaces include console, filesystem, system, time, threading, dynamic-library, networking, regular-expression, and clipboard services. Platform implementations include Win32, POSIX, DOS, and microcontroller-oriented variants.

libkernel provides VMContext, lexical services, memory management, security context handling, BIOS virtualization, virtual devices, and virtual console support.

libengine contains the parser, AST evaluator, runtime functions, variables, strings, and VM execution loop.

libhardware contains segmented memory support, graphics rasterization, and hardware emulation interfaces.

libserver provides virtual networking, protocol handling, background tasks, virtual filesystem support, cryptographic services, and regular-expression functionality.

libscript provides non-interactive script execution and script-oriented file operations.

libcore provides the foundational REPL, numeric formatting, PRINT support, metadata registration, and related runtime services.

libflex provides optional metaprogramming facilities including ALIAS, OVERRIDE, and SCOPE, together with module loading and optional language extensions.

libstandard provides the standard terminal workstation environment, including the TUI editor and debugging services.

libadvanced provides desktop graphics, multimedia, SDL2, and OpenGL integration.

libext provides the extension boundary for user and third-party components.

The executable targets select subsets of this library structure.

The baspp target is the standard desktop environment. It includes the advanced library and its associated graphics and workstation functionality. Its documented default memory allocation is 640 MB.

The bpp target provides a reduced REPL environment without the desktop graphics and multimedia components. Its documented default memory allocation is 384 MB.

The bs target provides a non-interactive script execution environment. Its documented default memory allocation is 64 MB.

1. SELECTIVE COMPILATION

A specific executable can be built without rebuilding unrelated targets.

For a Windows MSVC build, the standard desktop executable can be selected with:

```
cmake --build build_win --target baspp --config Release
```

For a Linux build:

```
cmake --build build_linux --target baspp
```

The headless script runner can be built independently:

```
cmake --build build_win --target bs --config Release
```

The selected target determines which library objects and optional subsystems are included in the executable.

1. ERROR HANDLING

BASIC++ errors are represented by BppError values. A new error must first be assigned an enumeration value in engine/include/types/errors.h.

```
typedef enum {
    /* Existing errors. */

    ERR_CUSTOM_ERROR = 95
} BppError;
```

The corresponding diagnostic text is registered in engine/src/types/errors.c.

```
case ERR_CUSTOM_ERROR:
    return "Custom feature initialization failed";
```

Runtime failures must be propagated through the VM error mechanism. vm_set_error() records the error state and returns control to the execution loop.

Argument count failures, invalid data types, invalid numeric ranges, and permission failures must produce the corresponding BASIC++ runtime error rather than allowing invalid data to propagate into implementation code.

Error 5 represents ERR_ILLEGAL_FUNCTION_CALL. It is used for invalid parameter ranges and other illegal runtime arguments where specified by the affected language component.

Error 13 represents ERR_TYPE_MISMATCH. It is used when an operand cannot be converted or interpreted according to the function or statement's defined type requirements.

Error 70 represents ERR_PERMISSION_DENIED where a security capability check prevents execution.

1. HELP AND CATALOG REGISTRATION

Interactive documentation is registered through engine/src/runtime/help_data.h. A keyword's help entry is made available to the HELP and CATALOG facilities through the runtime metadata registry.

A new language component should therefore have both a plain-text help document and a runtime metadata entry. Documentation should describe syntax, argument requirements, return values, runtime errors, implementation location, and relevant compatibility information.

1. MEMORY MANAGEMENT

Runtime allocations are controlled through the BASIC++ memory subsystem. Components must not create independent allocation systems that bypass the memory context unless the component explicitly defines such an interface.

Temporary buffers must be initialized before use. calloc() or an equivalent zero-initialization mechanism is required where the data structure depends on an initially cleared state.

Memory resizing must preserve the original allocation if the expansion operation fails. Code must not overwrite the only valid pointer with a failed realloc() result.

String ownership is explicitly tracked. Every retain operation must have a corresponding release operation along every execution path.

The implementation must not access memory through an invalidated pointer after a failed allocation or resize operation.

1. SECURITY CONTEXT

Operations that access protected resources are subject to SecurityContext capability checks. The defined capability flags include CAP_IO, CAP_FS, and CAP_SYS.

A language extension must not bypass the security context by accessing operating-system services directly. Filesystem, device, process, and other privileged operations must pass through the appropriate platform abstraction and security checks.

1. PORTABILITY REQUIREMENTS

The engine is written for ISO C17.

Pointer values must not be converted directly to ordinary integer types. When a pointer must be represented as an integer, uintptr_t or intptr_t from stdint.h must be used.

The engine source is intended to compile without operating-system headers in the core language modules. Host-specific code belongs in the platform abstraction layer.

Console input and output for the affected BASIC++ runtime components is restricted to 7-bit ASCII where specified by the project configuration. UTF-8 and other multibyte terminal encodings must not be introduced into components that require the ASCII interface.

The implementation must remain valid on both 32-bit and 64-bit targets. Numeric and bitwise operations must not depend on host byte order.

The compiler warning policy is intended to remain clean under GCC and Clang with:

```
-Wall -Wextra -Werror
```

and under MSVC with:

```
/W4 /WX
```

POSIX targets may define:

```
_POSIX_C_SOURCE=200809L
```

when required by the platform abstraction build.

Windows builds may use delay loading for optional SDL2 components so that executables without the optional graphics environment can start without an immediate dynamic-link failure.

1. DEBUGGING

A runtime failure should first be reproduced using the BASIC++ self-test system.

```
baspp -c "SELFTEST"
```

Interactive tracing can be enabled with TRON or DEBUG ON where supported by the active runtime.

The runtime log files baspp.log, bpp.log, and bs.log can be examined for initialization and execution failures.

A native debugger such as GDB or Visual Studio can be attached to the executable. A breakpoint should normally be placed in the statement or function handler responsible for the failing operation.

When debugging parser failures, inspect the token stream returned by lex_next(). Confirm that identifiers, keywords, numbers, and strings are being classified correctly.

When debugging value failures, inspect BValue.type before examining the union payload.

When debugging strings, trace str_retain() and str_release() calls and verify that every owned reference is released exactly once.

Undefined references during GCC linking should be investigated against the CMake target dependency graph and linker group configuration.

POSIX compilation problems involving unavailable interfaces should be checked against the target's feature-test definitions.

DOCS_AUDIT.md and the repository issue history should be checked when behavior changed after a recent source modification.

1. REQUIRED SOURCE HEADER FORMAT

Every C source and header file under engine/src and engine/include must begin with a project-specific technical header.

The header must identify the component's purpose, architectural role, implementation method, dependencies, compilation target, executable inclusion rules, extension points, immutable interfaces, expected runtime behavior, diagnostic procedure, initialization requirements, portability requirements, and prerequisite source and header files.

The header is intended to describe the implementation contract rather than provide general project commentary. It should contain information that allows a developer to understand what the source file owns, what it depends upon, what it may modify, and what assumptions are made by callers.

1. EXTENDING THE RUNTIME

New language functionality should be implemented as an isolated module whenever practical. The implementation should expose only the interfaces required by the parser, evaluator, VM, or registration system.

A new module should not modify unrelated engine components merely to simplify registration. Registration tables, metadata structures, and CMake target definitions should provide the integration boundary.

New platform functionality must be implemented through libplatform. New virtual hardware should use the VDev interface. New graphics functionality should use the graphics abstraction rather than embedding host-specific display calls in the BASIC interpreter.

New compiler output support can be added through the bppc backend architecture. Existing C17 and WebAssembly backends can be extended independently when the corresponding target is enabled.

1. INVARIANTS

The following implementation requirements are fixed parts of the BASIC++ architecture.

The engine must remain C17 code.

The VM execution model must remain non-recursive with respect to host C stack consumption.

The platform abstraction boundary must not be bypassed by core engine code.

BValue union members must only be accessed after type validation.

String ownership must remain balanced across successful and failed execution paths.

Pointer-to-integer conversions must use uintptr_t or intptr_t.

The documented executable memory configurations must remain unchanged unless the target specification itself is deliberately revised.

The boot sequence and rollback order must remain synchronized.

The source-file naming convention for language components must remain consistent with the keyword or component registration system.

1. FUTURE IMPLEMENTATION AREAS

The existing architecture permits additional virtual devices, language extensions, metaprogramming features, compiler backends, graphics profiles, terminal editor functionality, WebAssembly builds, embedded targets, networking interfaces, and user-defined type support.

These additions should be implemented as separate modules where possible. Existing parser, evaluator, VM, platform, and security interfaces should be reused rather than duplicated. A new feature should not introduce a second memory manager, parser, platform interface, or unrelated runtime dispatch mechanism.

The primary requirement for future extensions is that they remain compatible with the existing execution model, memory ownership rules, platform abstraction boundary, and C17 implementation requirements.